

OpenAI Debugging — Worked Example: Solution

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

- [Round 1: Basic](#)
- [Round 2: Intermediate](#)
- [Round 3: Real-world](#)
- [Round 4: Multi-agent](#)
- [Real cause vs. symptom fix](#)

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Basic

Real cause: asking a chat model to "reply with JSON" in a plain prompt is a request, not a guarantee — the model is still free to add a friendly sentence around it, and often does.

`json.loads()` has no tolerance for anything before the first `{`.

The fix:

```
response = client.responses.parse(  
    model="gpt-4o-mini",  
    input=user_text,  
    text_format=Summary,  
)  
data = response.output_parsed
```

Switching to OpenAI's structured-output feature (`text_format=`) constrains what the model can produce at generation time, instead of hoping the model's free-form text happens to parse.

Test that would have caught it:

```
def test_summary_extraction_returns_a_summary_object():  
    result = summarize("Some article text about a product launch.")  
    assert isinstance(result, Summary)
```

Asserting on the *type* of the result, not just that the call didn't raise, catches both this bug and the "valid JSON, wrong shape" case Doc02's Core Concepts warns about.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Intermediate

Real cause: OpenAI's structured-output "strict" mode builds a JSON Schema straight from the Pydantic model, and strict mode does not allow `default` values in that schema — every field has to be genuinely required, with `None` handled through `Optional / Union`, not a Python default.

`tags: list[str] = []` is a perfectly normal Pydantic model on its own, but it isn't a valid strict-mode schema, so the API rejects the whole request before generation starts.

The fix:

```
class Summary(BaseModel):
    summary: str
    tags: list[str]
```

Drop the default. If "no tags" is a real, valid answer, have the model return an empty list explicitly — the schema now just requires the field to be present, not that it be non-empty.

Test that would have caught it:

```
def test_summary_schema_has_no_defaults():
    schema = Summary.model_json_schema()
    for field_name, field_schema in schema["properties"].items():
        assert "default" not in field_schema
```

This test catches the mistake before a real API call is even needed — checking the schema OpenAI will actually receive is faster and cheaper than waiting for a `BadRequestError` from a live call.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Real-world

Real cause: nothing in the chatbot ever checks the running token count before sending. Every turn just appends to history and resends the whole thing. The failure isn't caused by any one bad message — it's caused by the cumulative total quietly growing every turn until one particular turn happens to tip it over 128,000 tokens.

The fix:

```
def build_messages_within_budget(history, system_prompt, max_tokens=120000):
    messages = [system_prompt]
    running_total = count_tokens(system_prompt)

    kept_in_order = []
    for message in reversed(history):
        message_tokens = count_tokens(message)
        if running_total + message_tokens > max_tokens:
            break
        kept_in_order.append(message)
        running_total += message_tokens

    kept_in_order.reverse()
    messages.extend(kept_in_order)
    return messages
```

Walking the history from the most recent message backward, stopping once the budget would be exceeded, keeps the most relevant (recent) turns and drops the oldest ones first — instead of failing outright once the total crosses the line.

Test that would have caught it:

```
def test_long_session_stays_within_token_budget():
    history = build_fake_history(turn_count=200)
    messages = build_messages_within_budget(history, SYSTEM_PROMPT, max_tokens=120000)
    total = 0
    for message in messages:
        total += count_tokens(message)
    assert total <= 120000
```

A test with only 2-3 turns can't catch this — it has to simulate a genuinely long session, the same way the bug only showed up in production after many real turns.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Multi-agent

Real cause: every agent writing its full scratchpad into shared state means each downstream agent's prompt is built from everyone-before-it's raw internal reasoning, not their finished conclusions. This is exactly the shared-state design mistake Doc11's Core Concepts calls out by name — too much in shared state buries what a later agent actually needs under noise it has to wade through, and occasionally pushes the whole request over the same token budget as Round 3, just sourced from 4 agents' scratchpads instead of one long conversation.

The fix:

```
class SharedState(TypedDict):
    task: str
    research_findings: str  # Research agent's finished output only
    analysis: str           # Analysis agent's finished output only
    draft: str              # Writer agent's finished output only

def research_node(state):
    raw_scratchpad = run_research_tools(state["task"])
    finished_summary = summarize_findings(raw_scratchpad)
    # raw_scratchpad stays local to this node – never written to shared state
    return Command(goto="analysis_agent", update={"research_findings": finished_s
```

Each node keeps its own tool-call scratchpad local and only ever writes its *finished* output into shared state — the same "what actually belongs here" question Doc11 asks for every field.

Test that would have caught it:

```
def test_shared_state_never_contains_raw_scratchpads():
    result_state = run_full_pipeline(sample_task)
    for field_name, field_value in result_state.items():
        assert "tool_call_log" not in field_name
        assert len(field_value) < 4000
```

Checking the *size and shape* of what ends up in shared state after a real end-to-end run — not just whether the pipeline finished without an exception — is what catches quality degradation and occasional context blowups that a small, hand-built fake state for one agent can't reproduce.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Real cause vs. symptom fix

A symptom fix would be catching `JSONDecodeError` and retrying with a slightly different prompt hoping for cleaner output (Round 1), catching the schema error and just removing the `tags` field entirely instead of understanding strict mode's actual rule (Round 2), catching the context-length error and silently dropping the oldest half of history with no real budget logic (Round 3), or capping shared-state field *string length* with a blunt truncation instead of deciding what belongs there at all (Round 4). Each of those makes today's specific failure quiet without addressing why it happens, which is exactly Core Concepts' point about a real-cause fix being checked by actually understanding it, not by the symptom going away — a truncation hack, in particular, tends to cut off content at an arbitrary point instead of the *right* point, trading one confusing failure for a quieter, harder-to-notice one.